

Project Report – Phase 1 and Phase 2

Design, Data Quality, Cleaning, and ETL of the APP_SNAPSHOT Data Mart

1 Introduction

This report presents the first two phases of the Google Play Store data warehouse project. Phase 1 focuses on the conceptual and logical design of the data mart. Starting from the reconciled relational schema, we identify the fact of interest, build and edit the attribute tree, define dimensions and measures, derive the Dimensional Fact Model (DFM) schema, and translate the conceptual schema into a logical star schema for ROLAP analysis.

Phase 2 focuses on data management. It documents the population of the reconciled database, the baseline data quality assessment, the cleaning and audit strategy, and the ETL process from the clean/reconciled layer to the data warehouse. The final solution is centered on the fact APP_SNAPSHOT, implemented as a star schema extended with a weighted bridge table for the many-to-many relationship between app snapshots and Genre.

2 Data Source and Preliminary Requirement Analysis

2.1 Data Source

The project is based on a Google Play Store dataset containing information about mobile applications published in the store. The data includes both descriptive information about applications and quantitative attributes observed at snapshot level, such as rating, number of reviews, number of installs, price, size, and last update date.

From an analytical point of view, this source is appropriate for multidimensional design because it provides:

- descriptive attributes that can be organized into dimensions, such as app, category, app type, content rating, and genre;
- numerical attributes that can be analyzed as measures;
- a temporal business attribute (`last_updated_date`) supporting time-based analysis;
- a non-trivial structural aspect, namely the many-to-many association between snapshots and genres.

2.2 Preliminary Requirement Analysis

The goal of the data mart is to support exploratory and aggregate analysis of the observed state of applications in the store. In particular, the design is intended to support questions such as:

- how ratings, reviews, and installs vary across categories;
- how free and paid apps differ in price and popularity;

- how content rating is associated with the main numerical indicators;
- how application snapshots can be analyzed over time through the update date;
- how genre-based analyses can be performed despite the many-to-many relationship with app snapshots.

These requirements motivate the choice of a snapshot-oriented fact and a star schema suitable for OLAP-style aggregation and slicing.

3 Reconciled Source Schema and Data-Driven Design Approach

3.1 Re-engineering Step

The original source data was first reorganized into a reconciled relational schema. This re-engineering step makes the source more suitable for subsequent data warehouse design by separating descriptive entities, identifying relationships explicitly, and preparing the data for the data-driven conceptual modeling process.

The reconciled database was implemented in Supabase PostgreSQL. This report starts from that reconciled layer and derives the conceptual and logical schemas of the data mart.

3.2 Schema of the Reconciled Database

Figure 1 shows the reconciled relational schema used as input for the data-driven design. The central relation is `reconciled.app_snapshot`, which stores the snapshot-level numerical attributes and business descriptors. The descriptive tables `reconciled.app`, `reconciled.category`, `reconciled.app_type`, and `reconciled.content_rating` provide the main candidates for dimensions. Moreover, the association `reconciled.app_snapshot_genre` captures the many-to-many relationship between snapshots and genres, which later motivates the multiple arc in the DFM schema and the bridge table in the logical design.

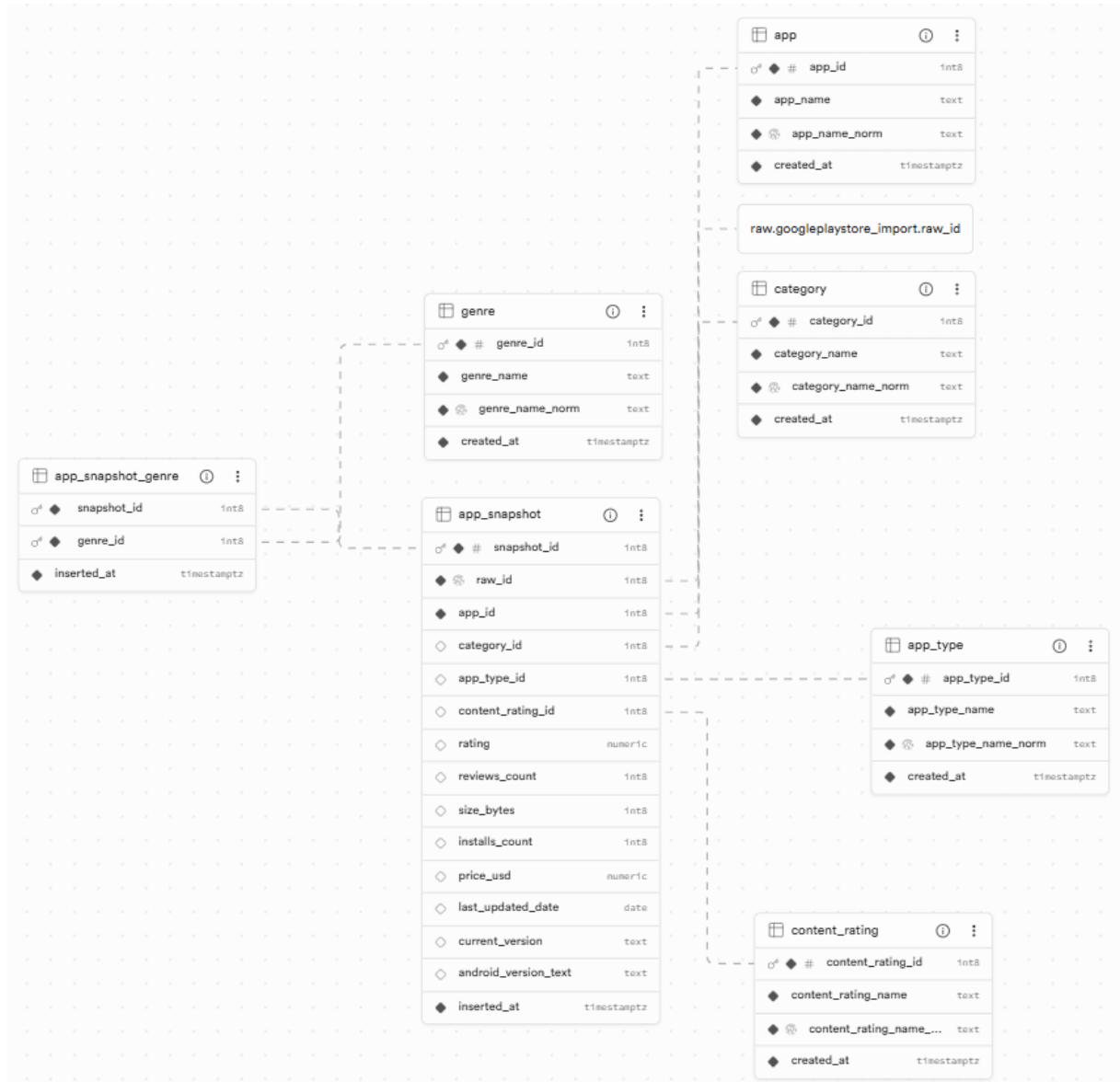


Figure 1: Reconciled relational schema used as input for the data-driven design.

The reconciled schema is implemented in Supabase PostgreSQL and represents the starting point for the data-driven workflow.

3.3 Data-driven Design Workflow

The design follows the data-driven workflow adopted in class:

1. definition of the fact;
2. construction of the basic attribute tree;
3. editing of the attribute tree;
4. definition of dimensions;
5. definition of measures;
6. design of the final fact schema;
7. translation into a logical star schema.

4 Conceptual Design

4.1 Definition of the Fact

The fact chosen for the analysis is `APP_SNAPSHOT`. This choice is motivated by the nature of the relation: it represents a dynamic phenomenon of interest for decision-making, namely the observed state of an application at a specific observation point. Each record contains quantitative properties such as rating, number of reviews, number of installs, price, and size, which make the relation suitable for multidimensional analysis.

In contrast, tables such as `app`, `category`, `app_type`, and `content_rating` contain mostly descriptive and relatively stable data, and are therefore more appropriate as dimensions.

At conceptual level, the grain of the fact can be described as one observed snapshot of one application at one update date.

4.2 Construction of the Basic Attribute Tree

Starting from the reconciled schema, the basic attribute tree is built by taking the identifier of `APP_SNAPSHOT` as the root and recursively following attributes and to-one associations. The resulting tree includes:

- direct numerical attributes of `app_snapshot`: `rating`, `reviews_count`, `installs_count`, `price_usd`, `size_bytes`;
- business attributes such as `last_updated_date`, `current_version`, and `android_version_text`;
- to-one related descriptive entities: `app`, `category`, `app_type`, and `content_rating`.

The relation `app_snapshot_genre` is not part of the basic attribute tree because it represents a to-many association. Therefore, it does not belong to the basic recursive construction based only on functional dependencies and to-one associations.

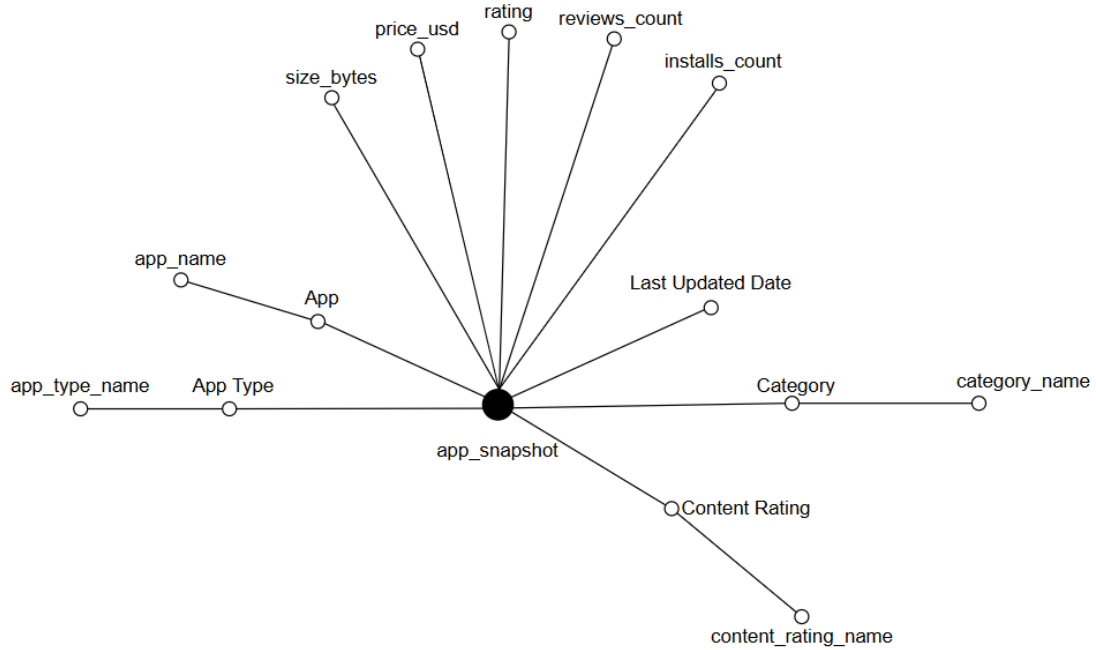


Figure 2: Basic attribute tree of `APP_SNAPSHOT`. The tree includes the direct numerical attributes, the business date `last_updated_date`, and the descriptive entities reachable through to-one associations. The to-many association with `Genre` is intentionally excluded at this stage.

4.3 Editing of the Attribute Tree

The attribute tree is then edited through pruning and grafting operations.

Pruned elements. The following technical attributes are removed because they are not useful for aggregation at the conceptual level:

- `snapshot_id` in the conceptual representation;
- `raw_id`;
- the technical identifiers `app_id`, `category_id`, `app_type_id`, and `content_rating_id` as labels of intermediate nodes.

In addition, `current_version` and `android_version_text` are pruned from the conceptual schema because they are not used to define aggregation paths in the current analysis scenario.

Grafted elements. The technical identifiers of related tables are replaced by business dimensions and their descriptive leaves:

- App $\rightarrow$ `app_name`
- Category $\rightarrow$ `category_name`
- App Type $\rightarrow$ `app_type_name`
- Content Rating $\rightarrow$ `content_rating_name`

Time dimension. The business attribute `last_updated_date` is transformed into the temporal dimension `Date`. The day-level member is the full calendar date, which can be rolled up to month and year:

`full_date` $\rightarrow$ `month` $\rightarrow$ `year`

In the logical schema, the attribute `day` is stored as the day-of-month descriptor of `full_date`; it is not interpreted as an independent determinant of month and year.

Advanced construct. The association with **Genre** is reintroduced at the conceptual level as a multiple arc. This modeling choice is required because the reconciled schema shows that the same app snapshot may be associated with more than one genre.

4.4 Definition of Dimensions

After editing the attribute tree, the selected dimensions are:

- App
- Category
- App Type
- Content Rating
- Date

These dimensions determine the granularity of the primary events represented by the fact. The **Genre** dimension is modeled separately through a multiple arc because its relationship with the fact is many-to-many.

4.5 Definition of Measures

The measures are the numerical attributes directly associated with each snapshot:

- `rating`
- `reviews_count`
- `installs_count`
- `price_usd`
- `size_bytes`

From the point of view of additivity, `rating`, `price_usd`, and `size_bytes` are treated as non-additive measures. The measures `reviews_count` and `installs_count` are snapshot-based measures and therefore must be handled carefully along the temporal hierarchy, since summing them across time for the same app may lead to double counting of successive observations.

4.6 Final DFM Schema

The final conceptual schema is centered on the fact `APP_SNAPSHOT`. The dimensions and hierarchies are:

- App $\rightarrow$ `app_name`
- Category $\rightarrow$ `category_name`
- App Type $\rightarrow$ `app_type_name`
- Content Rating $\rightarrow$ `content_rating_name`
- Date: `full_date` $\rightarrow$ `month` $\rightarrow$ `year`; the logical attribute `day` represents the day of month

The measures are `rating`, `reviews_count`, `installs_count`, `price_usd`, and `size_bytes`.

The final DFM schema also includes an advanced construct: **Genre** is represented as a *multiple arc*. This modeling choice is required because the reconciled schema contains the relation `app_snapshot_genre`, and data inspection confirmed that the same snapshot may be associated with more than one genre.

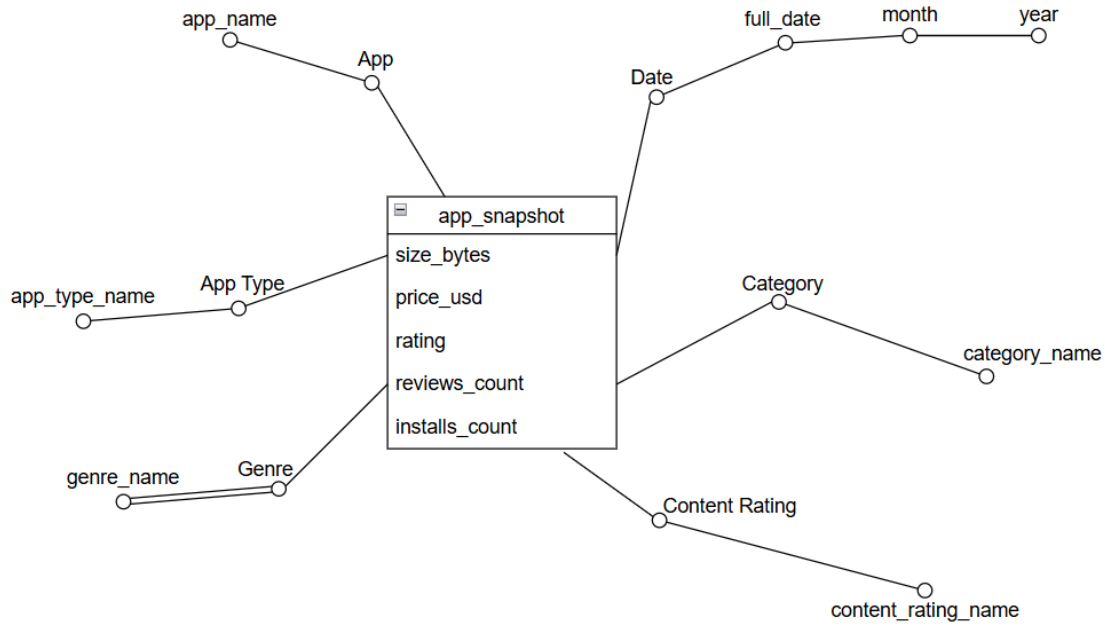


Figure 3: Final DFM schema for APP_SNAPSHOT. The figure shows the fact, the selected dimensions and hierarchies, and the multiple arc with **Genre**.

5 Logical Design

5.1 Choice of the Logical Model

The conceptual schema is translated into a ROLAP logical model. The chosen implementation is a star schema. Since the conceptual schema contains a multiple arc involving **Genre**, the final logical solution is a star schema extended with a bridge table.

The logical schema is also designed for implementation in Supabase PostgreSQL.

5.2 Dimension Tables

The following dimension tables are created:

- `dim_app(app_key, app_id, app_name)`
- `dim_category(category_key, category_id, category_name)`
- `dim_app_type(app_type_key, app_type_id, app_type_name)`
- `dim_content_rating(content_rating_key, content_rating_id, content_rating_name)`
- `dim_last_updated_date(last_updated_date_key, full_date, day, month, quarter, year)`
- `dim_genre(genre_key, genre_id, genre_name)`

Each dimension table is identified by a surrogate key and stores the business key coming from the reconciled schema. At logical level, the conceptual dimension **Date** is implemented through the table `dim_last_updated_date`, in order to preserve traceability to the source attribute `last_updated_date`.

5.3 Fact Table

The fact table is:

```
fact_app_snapshot(app_snapshot_key, snapshot_id, app_key, category_key,
app_type_key,
content_rating_key, last_updated_date_key, rating, reviews_count, installs_count,
price_usd, size_bytes)
```

The grain of the fact table is one row for each row of `reconciled.app_snapshot`. The attribute `snapshot_id` is retained as a business key to preserve traceability with the reconciled layer, while `app_snapshot_key` is the surrogate primary key of the fact table.

5.4 Bridge Table for the Multiple Arc

The many-to-many relationship between app snapshots and genres is implemented through the bridge table shown in Figure 4:

```
bridge_app_snapshot_genre(app_snapshot_key, genre_key, weight)
```

The primary key of the bridge table is the pair (`app_snapshot_key`, `genre_key`). The attribute `weight` is used to support weighted analyses over the multiple arc. In the implemented loading workflow, the weight is computed as:

$$\text{weight} = \frac{1}{\text{number of genres associated with the snapshot}}$$

Data checks confirmed that:

- the minimum weight is 0.5;
- the maximum weight is 1.0;
- the sum of the weights for each `app_snapshot_key` is equal to 1.

5.5 Handling Missing Dimension Values

During the data management phase, one loaded snapshot was found to have a missing app type in the source data. To preserve the fact grain and avoid nullable foreign keys in the warehouse fact table, the logical loading process introduces a technical `Unknown` member in `dim_app_type`. This choice does not change the conceptual design; it only affects the ETL strategy and makes the data warehouse robust to incomplete source records. No technical `Unknown` member was required for `Content Rating` or `Date`, because the loaded reconciled snapshots contain valid values for these dimensions.

5.6 Indexes and Constraints

The implemented logical schema includes:

- primary keys on all dimension tables and on the fact table;
- a unique constraint on `fact_app_snapshot.snapshot_id`;
- foreign key constraints from the fact table to the dimensions;
- a primary key on `bridge_app_snapshot_genre(app_snapshot_key, genre_key)`;
- a NOT NULL and CHECK constraint on the bridge attribute `weight`;
- indexes on the main foreign keys to improve join performance.

5.7 Glossary of the Measures

Table 1 provides a glossary of the measures used in the data mart.

Table 1: Glossary of the measures.

Measure	Meaning	Unit	Aggregation behavior
rating	Observed average user rating of the app at snapshot time.	score	Non-additive. Typically analyzed through averages rather than sums.
reviews_count	Number of user reviews observed for the app at snapshot time.	count	Snapshot-based. It can be aggregated across non-temporal dimensions, but summing it across time for the same app may double count successive observations.
installs_count	Number of installs observed for the app at snapshot time.	count	Snapshot-based. It can be aggregated across non-temporal dimensions, but summing it across time for the same app may double count successive observations.
price_usd	Observed app price at snapshot time.	USD	Non-additive. Typically analyzed through averages, minima, maxima, or distributions.
size_bytes	Observed app size at snapshot time.	bytes	Non-additive. Typically analyzed through averages, minima, maxima, or distributions.



Figure 4: Implemented data warehouse star schema in Supabase PostgreSQL. The schema extends the logical star schema with data quality flags and a weighted bridge table for the many-to-many relationship with Genre.

6 Phase 2: Data Management

6.1 Objective and Execution Workflow

The objective of Phase 2 is to populate the reconciled database, assess data quality, implement cleaning tasks, and design the ETL process from the clean/reconciled layer to the data warehouse. The implementation was carried out directly in Supabase PostgreSQL using SQL scripts. This

choice makes the process reproducible, auditable, and close to the final database environment.

The execution order of the implemented scripts is:

1. `01_reconciled_schema.sql`: creation of the raw and reconciled schemas;
2. import of `googleplaystore.csv` into `raw.googleplaystore_import`;
3. `02_domain_load.sql`: loading of domain/master tables;
4. `03_app_snapshot_etl.sql`: transformation of raw rows into typed app snapshots;
5. `04_genre_etl.sql`: extraction of genres and population of the snapshot-genre bridge;
6. `05_dqa_queries.sql`: baseline data quality assessment;
7. `06_cleaning_and_audit.sql`: creation of the clean layer, quality flags, audit log, and weighted genre bridge;
8. `07_dw_star_schema.sql`: creation of the data warehouse star schema;
9. `08_dw_etl.sql`: ETL from the clean layer to the data warehouse.

6.2 Population of the Reconciled Database

The original CSV file was first loaded into the raw table `raw.googleplaystore_import`. From this raw table, the reconciled schema was populated through SQL transformations. The loading process separates descriptive entities into domain/master tables and stores the numerical snapshot-level attributes in `reconciled.app_snapshot`. The many-to-many relationship between snapshots and genres is represented by `reconciled.app_snapshot_genre`.

Table 2 summarizes the reconciliation counts obtained after the raw-to-reconciled loading process.

Table 2: Reconciliation counts after loading the reconciled database.

Metric	Value
Raw rows	10,841
Loaded app snapshots	10,840
Rows not loaded to snapshot	1
Apps	9,638
Categories	33
App types	2
Content ratings	6
Genres	53
Snapshot-genre bridge rows	11,288

The single raw row not loaded into `reconciled.app_snapshot` corresponds to a shifted malformed source record. It was kept in the raw layer for traceability but excluded from the typed reconciled snapshot table.

6.3 Baseline Data Quality Assessment

A baseline data quality assessment was performed on the reconciled layer before cleaning. The checks cover the main operational data quality dimensions used during the course: completeness, uniqueness, validity, consistency, timeliness, and accuracy. Accuracy is documented as a limitation because a trusted external reference source is not available for verifying whether the Google Play Store values reflect real-world truth at the current time.

The most relevant baseline results are shown in Table 3. Structural and validity checks are green, while missing ratings and missing/variable sizes are warning-level issues. These warning-level issues are analytically meaningful and therefore should not be blindly imputed.

Table 3: Selected baseline DQA results on the reconciled layer.

Dimension	Metric	Checked	Issues	Severity
Completeness	Missing rating	10,840	1,474	Yellow
Completeness	Missing or variable size	10,840	1,695	Yellow
Completeness	Missing app type	10,840	1	Green
Completeness	Missing current version	10,840	8	Green
Completeness	Missing Android version	10,840	2	Green
Uniqueness	Duplicate app normalized name	9,638	0	Green
Uniqueness	Duplicate raw id in snapshot	10,840	0	Green
Uniqueness	Duplicate snapshot-genre pair	11,288	0	Green
Validity	Invalid domain values after load	41	0	Green
Consistency	Type-price conflicts	10,840	0	Green
Timeliness	Missing or future update dates	10,840	0	Green

6.4 Cleaning Strategy

The cleaning strategy is conservative. The goal is not to force every missing value to become a value, but to distinguish between data errors and analytically meaningful missing information. For this reason, the pipeline uses the following decisions:

- missing ratings are preserved as `NULL` and marked with `rating_missing_flag`;
- missing sizes are preserved as `NULL` and marked with `size_missing_flag`;
- values equal to `Varies with device` are explicitly identified by `size_varies_with_device_flag`;
- the single missing app type is preserved and marked with `app_type_missing_flag`;
- missing descriptive version attributes are standardized to `NULL` and flagged;
- the many-to-many genre relationship is preserved and handled through a weighted bridge instead of being collapsed to a single genre.

This strategy avoids biased imputation of analytical measures such as `rating` and `size_bytes`. It also makes data quality transparent to analysts and dashboard users through explicit flags.

6.5 Clean Layer and Audit Log

The script `06_cleaning_and_audit.sql` creates the schema `clean`. The main clean tables are `clean.app_snapshot_clean`, `clean.app_snapshot_genre_clean`, and `clean.cleaning_audit_log`.

The clean snapshot table preserves the same grain as the reconciled snapshot table: one row per app snapshot.

Table 4 reports the main cleaning summary.

Table 4: Cleaning summary.

Metric	Value
Clean app snapshot rows	10,840
Clean bridge rows	11,288
Audit log rows	3,628
Rating missing flag rows	1,474
Size missing flag rows	1,695
Size varies with device flag rows	1,695
App type missing flag rows	1
Current version missing flag rows	8
Android version missing flag rows	2
Type-price conflict flag rows	0
Invalid measure flag rows	0
Genre missing flag rows	0
Multiple genre flag rows	448
DQ status OK rows	7,347
DQ status flagged OK rows	3,493
DQ status error review rows	0

The audit log records the cleaning or flagging action applied to each affected row. Table 5 summarizes the audit log by issue type.

Table 5: Audit summary by issue type.

Issue type	Action	Rows
size_varies_with_device	PRESERVE_AND_FLAG	1,695
rating_missing	PRESERVE_AND_FLAG	1,474
multiple_genre_membership	PRESERVE_AND_USE_WEIGHTED_BRIDGE	448
current_version_missing	STANDARDIZE_NULL_AND_FLAG	8
android_version_missing	STANDARDIZE_NULL_AND_FLAG	2
app_type_missing	PRESERVE_AND_FLAG	1

The after-cleaning scorecard confirms that all managed missing values are explicitly flagged, no invalid measures remain, no type-price conflicts are present, no snapshots are missing genres, and the clean row count matches the reconciled row count.

6.6 Handling the Many-to-Many Relationship with Genre

The baseline genre distribution confirms that **Genre** is a real many-to-many analysis dimension. Table 6 shows that 448 snapshots are associated with two genres.

Table 6: Number of genres per app snapshot.

Genre count	Snapshot count
1	10,392
2	448

A direct join between the fact table and the genre bridge would replicate the 448 snapshots with two genres and could lead to double counting. To prevent this issue, `clean.app_snapshot_genre_clean` computes a bridge weight as:

$$\text{weight} = \frac{1}{\text{number of genres associated with the snapshot}}$$

Thus, a snapshot with one genre receives weight 1, while a snapshot with two genres contributes 0.5 to each genre. The bridge weight check returned zero failing rows, meaning that the sum of weights for each snapshot is equal to 1. This directly supports safe OLAP analysis over **Genre**.

6.7 Data Warehouse Star Schema Implementation

The logical star schema designed in Phase 1 was implemented in the schema `dw`. It contains the following tables:

- dimensions: `dim_app`, `dim_category`, `dim_app_type`, `dim_content_rating`, `dim_last_updated_date`, `dim_genre`;
- fact: `fact_app_snapshot`;
- bridge: `bridge_app_snapshot_genre`;
- analysis views: `v_fact_app_snapshot_enriched`, `v_genre_fractional_analysis`, `v_genre_membership`, and `v_genre_weighted_kpis`.

A technical `Unknown` member is added to `dim_app_type` to preserve referential integrity for the single snapshot with missing app type. The fact table also includes data quality flags from the clean layer so that dashboard users can filter or explain missing values.

6.8 ETL from the Clean Layer to the Data Warehouse

The script `08_dw_etl.sql` populates all dimensions, the fact table, and the genre bridge from the clean and reconciled layers. Table 7 shows the final data warehouse load summary.

Table 7: Data warehouse load summary.

Metric	Value
dim_app rows	9,638
dim_category rows	33
dim_app_type rows	3
dim_content_rating rows	6
dim_last_updated_date rows	1,377
dim_genre rows	53
fact_app_snapshot rows	10,840
bridge_app_snapshot_genre rows	11,288
Unknown app type fact rows	1
Fact rating missing flag rows	1,474
Fact size missing flag rows	1,695
Fact multiple genre flag rows	448

Integrity checks were executed after the ETL. The results are shown in Table 8.

Table 8: Data warehouse integrity checks.

Check	Expected	Actual	Status
Fact rows match clean rows	10,840	10,840	PASS
Bridge rows match clean bridge rows	11,288	11,288	PASS
Genre bridge weight errors	0	0	PASS
Duplicate fact snapshot ids	0	0	PASS
Fact error-review rows	0	0	PASS

The data warehouse therefore preserves the clean-layer grain, maintains referential integrity, and provides safe support for genre-based analyses.

6.9 Analysis Views for OLAP and BI

To support correct analysis and reduce the risk of wrong aggregations, several views are created in the data warehouse schema. The most important one is `dw.v_genre_fractional_analysis`, which exposes weighted measures such as `weighted_reviews_count`, `weighted_installs_count`, and `weighted_snapshot_count`. These fields should be used when aggregating by `Genre`.

The view `dw.v_genre_membership_analysis` supports tag-based analysis, where one snapshot may legitimately be counted under more than one genre. In that case, totals across genres are not expected to match the total number of snapshots. This distinction is explicitly documented to prevent double counting in later dashboards.

7 Final Result of Phase 1 and Phase 2

The first two project phases produce the following outputs:

1. the identification of `APP_SNAPSHOT` as the fact of interest;
2. the construction and editing of the attribute tree;

3. the final DFM schema with dimensions, measures, and the multiple arc for **Genre**;
4. the logical translation into a star schema with a weighted bridge table;
5. the implementation and population of the raw and reconciled schemas in Supabase PostgreSQL;
6. a baseline data quality assessment on the reconciled layer;
7. a conservative cleaning strategy based on preservation, explicit flags, and audit logging;
8. a clean layer preserving the same fact grain as the reconciled snapshot table;
9. the implementation of the data warehouse star schema;
10. the ETL process from the clean/reconciled layer to the data warehouse;
11. integrity checks confirming that fact rows, bridge rows, and genre weights are correct.

8 Conclusion

The design and data management phases of the Google Play Store data mart are complete. The conceptual design follows the data-driven DFM methodology, and the logical design translates the model into a relational star schema suitable for OLAP analysis. The implemented data warehouse is centered on **APP_SNAPSHOT** and is extended with a weighted bridge table to handle the many-to-many relationship with **Genre**.

The Phase 2 pipeline confirms that the reconciled database is populated, quality issues are identified, meaningful missing values are preserved and flagged, and all cleaning decisions are recorded in an audit log. The ETL process loads 10,840 cleaned app snapshots into the fact table and 11,288 snapshot-genre associations into the bridge table. All integrity checks pass, including the check that the sum of genre bridge weights is equal to 1 for every snapshot. The resulting warehouse is therefore ready to support the visualization and OLAP analysis activities of Phase 3.